

ACP Learning · 技术说明附录

供技术人员查阅。配套主文档：《交付说明·实现结果》 日期：2026-07-08 | 代码仓库：
github.com/Chloeyouyou/acp-learning

一、技术栈

层	选型	说明
后端	Python / FastAPI	单服务，REST API
前端	Vue 3 + Vite + vue-router	构建产物由后端同源托管，无跨域问题
数据库	SQLite (WAL 模式) + Alembic 迁移	试点规模够用；全校规模换 PostgreSQL，已预留
AI	DeepSeek API	导师对话；接口层可替换其他大模型
沙箱	Docker 一次性容器（本地降级 subprocess）	学生代码隔离运行
部署	单 Docker 镜像	前后端一体打包，一条命令启动

二、核心架构设计

1. 六阶段导师状态机（系统的核心资产）

「LLM 软引导 + 规则硬门控」双层设计：

- **LLM 层**负责自然对话、共情、提问引导；
- **规则层**负责硬约束：阶段跃迁条件（如「没定位到雷行不允许进入修复阶段」）、导师红线（任何提示级别都不给可直接抄的代码；绝不臆断没真实运行过的结果——学生每次运行的真实输出会自动注入对话上下文）。

这保证了引导质量不依赖 AI「自觉」，prompt 漂移不会突破教学底线。

2. 事件溯源 (Event Sourcing)

所有能力评估都是可回放、可审计的派生结果：

- 四张核心事实表 (`events` / `execution_events` / `capability_scores` / `knowledge_states`) schema 稳定不动；
- `execution_events`、`code_snapshots`、`session_messages` 三张表 **append-only** (只增不改不删)，构成完整的过程事实源；
- 能力画像、成长轨迹、课程进度、教师端总览**全部是派生视图，零冗余状态表**——将来换更聪明的评估模型 (遗忘曲线、知识追踪)，只改派生函数，不迁数据。

过程回放即由 `code_snapshots` (代码逐版) + `session_messages` (动作时间线) 按时间戳合并生成，并计算 diff 信号 (改动是否触及雷行 → 区分定向修改 / 盲改)。

3. 身份与越权防护

- 无密码 HMAC 签名 token: 登录 (学号) → 签发 token, 前端统一带 Authorization;
- 所有 `/students/{id}/*` 端点**只认 token、不信自报 id** (`require_self` 越权门)，杜绝 IDOR (改 URL 看别人数据)；
- 设计取舍: 防遍历不防冒充 (试点场景够用)，将来加 PIN 即可升级，不动架构。

4. 沙箱隔离

学生代码在一次性 Docker 容器中运行: `--network none` (断网)、`--read-only` (只读文件系统)、`--memory 128m`、`--pids-limit 64` (防 fork 炸弹)、`--cap-drop ALL`、非 root、代码走 stdin。已通过 6 类逃逸验证 (读库 / 偷密钥 / 出网 / 死循环 / fork 炸弹 / 写根目录)。无 Docker 环境自动降级 subprocess 模式 (仅限开发演示)。

5. 并发安全

- per-session 回合锁: 同一会话的写操作串行化，防止并发丢写与 submit 竞态；
- SQLite WAL + `busy_timeout`；线程池加大到 64。同步端点跑在 threadpool，30 人试点规模实测足够。

三、部署与运维

- **打包**: 单 Dockerfile，前端构建产物打进镜像，后端同源托管。
- **启动** (VPS 上)：

```
docker run -d --name acp --restart unless-stopped -p 8000:8000 \
-v /root/acp-data:/data -e DATABASE_URL=sqlite:///data/acp.db \
-e DEEPSEEK_API_KEY=... acp-learning
```

- **数据持久化**：数据库落在宿主机挂载目录（ /root/acp-data ）， 容器重建 / 版本升级数据不丢。
- **数据库迁移**：Alembic 管理，服务启动时自动 upgrade head ；老库自动纳管（stamp baseline 后升级）， 全新库 / 存量库两条路径均已验证。
- **健康检查**： GET /api/health 返回版本号与沙箱后端类型。
- **升级流程**： git pull → docker build → docker rm -f acp → docker run （数据卷不动）。完整文档见仓库 docs/部署到VPS.md 。

四、质量保障

- **自动化测试 106 个，全过**，覆盖：状态机跃迁、导师红线、沙箱逃逸（6 条）、越权防护（3 条）、画像计算、课程派生、教师端聚合等核心逻辑；每次改动全量回归。
- **安全审计**：做过两轮系统性审计（安全专项 + 架构全面体检）， 高优先级问题（IDOR、沙箱逃逸、并发竞态、画像虚高等）全部修复并有对应回归测试；审计报告在仓库 docs/审计/ 目录可查。
- **文档**：设计文档（ docs/design/ 12 篇）、审计报告、部署文档、逐日进展日志齐全，交接成本低。

五、已知边界（技术视角）

边界	现状	升级路径
数据库	SQLite（试点够用）	换 PostgreSQL，改一个连接串 + Alembic 迁移
沙箱	依赖宿主机 Docker；无 Docker 时降级 subprocess（隔离弱）	生产环境务必启用 Docker 后端（ ACP_SANDBOX=docker ）
身份	学号即身份，无密码（防遍历不防冒充）	加 PIN / 对接学校统一认证
语言	Python 单语言	沙箱与题库流水线按语言扩展
前端 Arena 组件	单文件较大（约 1500 行），功能正常	已列为独立重构里程碑